



Technische  
Universität  
Braunschweig

## BACHELOR THESIS

Write down your Title right here!

Fabian Alich  
(Matrikel)  
(Programme)

Institute of Operating Systems and Computer Networks  
Algorithms Division

**First reviewer**  
(Anon Anonymous, Institute of XY)

**Second reviewer**  
(Anon Anonymous, Institute of YX)

**Supervised by**  
(Anon Anonymous)  
(tba)

May 7, 2025



### **Statement of Originality**

This thesis has been performed independently with the support of my supervisor/s. To the best of the author's knowledge, this thesis contains no material previously published or written by another person except where due reference is made in the text.

Braunschweig, May 7, 2025

---



# Aufgabenstellung

This is where your “Aufgabenstellung” goes. You should get this from your supervisor!



# Abstract

Your abstract gives a short introduction to your thesis (context), the problem description, and a brief description of your results.

You may want to include a german abstract, too.





# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>1</b>  |
| 1.1      | Results . . . . .                                  | 1         |
| 1.2      | Related Work . . . . .                             | 1         |
| 1.2.1    | Delaunay Triangulations . . . . .                  | 1         |
| 1.2.2    | Flips in Triangulations . . . . .                  | 2         |
| 1.2.3    | degree constrained minimum spanning tree . . . . . | 3         |
| 1.3      | Overview . . . . .                                 | 4         |
| <b>2</b> | <b>Preliminaries</b>                               | <b>5</b>  |
| <b>3</b> | <b>Content</b>                                     | <b>7</b>  |
| <b>4</b> | <b>Experiments</b>                                 | <b>9</b>  |
| <b>5</b> | <b>Conclusion (and Future Work)</b>                | <b>11</b> |



## List of Figures

|     |                                                                                    |   |
|-----|------------------------------------------------------------------------------------|---|
| 1.1 | <i>A perspective view of a terrain</i> aus [2, p. 183] . . . . .                   | 2 |
| 1.2 | <i>Examples of edge move, edge rotation, and edge slide</i> aus [1, p. 70] . . . . | 3 |



## List of Tables



# 1 Introduction

You can write an introduction here!

## 1.1 Results

We proof that the TRAVELING SALESMEN PROBLEM(TSP) is NP-complete.

... that the \textsc{Traveling Salesmen Problem}(TSP) is ...

You can use abbreviations for your problem names to avoid redundancy. Try to use existing abbreviations from related work if possible.

## 1.2 Related Work

Das ist meine Übersicht zur Related work

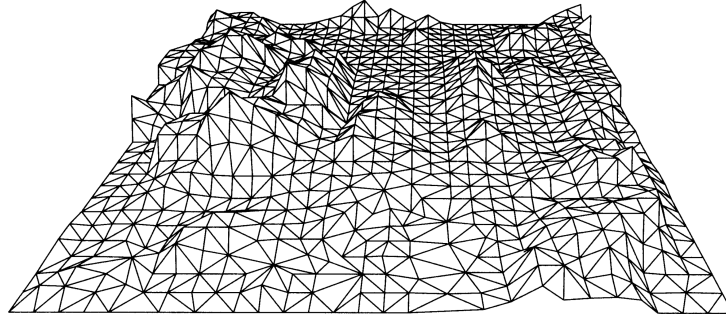
### 1.2.1 Delaunay Triangulations

Als Grundlage für den Algorithms können Delaunay Triangulation betrachtet werden. Diese Liefern eine Triangulation welche möglichst Gleichschenklige Dreiecke nutzt. In dem Buch von de Berg et.al. [2] wird diese Variante der Triangulation genauer betrachtet. In diesem Fall wird mit den Delaunay Triangulation das Problem gelöst, das höhendaten in einem zwei Dimensionalen Raum dargestellt werden.

Triangulation eignen sich aufgrund ihres Aufbaues gut um Höhendaten im zwei Dimensionalen Raum darzustellen (siehe Figure 1.1). Hier kann genutzt werden das Dreiecke verschiedene Innenwinkel und Kantenlängen haben können. Das Grundproblem ist hier das die höhendaten nur an bestimmten Punkten bekannt sind und somit die Restlichen Höhendaten erraten werden müssen.

- für die darstellung Höhen raten
- Gleichschenkligh sieht am besten aus
- ziel minimalen Winkel maximieren
- Formel für Kanten und Dreiecke bekommen
- drei Algos

Figure 1.1: A perspective view of a terrain aus [2, p. 183]



### 1.2.2 Flips in Triangulations

Flips sind Möglichkeiten, um Kanten in einer Triangulation auszutauschen. Da Triangulationen für die gleiche Knotenmenge, wie in [2] erwähnt, stets die gleiche Anzahl an Kanten besitzen, können keine Kanten hinzugefügt oder entfernt werden, um den Grad einer Triangulation zu verändern. Daraus folgt, dass für jede entfernte Kante eine andere Kante eingefügt werden muss. Eine besondere Art dieser Umstrukturierung von Kanten sind sogenannte Flips. Hurtado et al. [3] zeigten, dass jede Triangulation mindestens

$$\frac{n-4}{2}$$

flipbare Kanten besitzt, wobei  $n$  die Anzahl der Knoten bezeichnet. Laut dem Paper *Flips in planar graphs* [1] lässt sich jede mögliche Triangulation in jede beliebige andere Triangulation überführen, indem die notwendigen Flips durchgeführt werden. Daraus folgt, dass, wenn eine *degree-constrained triangulation* möglich ist, diese mit den nötigen Flips konstruiert werden kann.

Eine weitere Art von Flips sind sogenannte  $k$ -Flips. Diese stellen eine zusätzliche Möglichkeit dar, Kanten innerhalb einer Triangulation zu verschieben. Dabei werden  $k$  Kanten entfernt und durch  $k$  neue Kanten ersetzt. Normale Flips entsprechen somit 1-Flips. Der Vorteil von  $k$ -Flips liegt darin, dass mit einem Schritt größere strukturelle Veränderungen möglich sind.

#### **Absatz für Flips mit Binärbäumen hinzufügen, falls verwendet.**

Drei beispielhafte Flips sind in Figure 1.2 dargestellt. Zum einen der *edge move*, hierbei handelt es sich um das einfache Entfernen und anschließende Einfügen einer Kante. Etwas stärker eingeschränkt ist die *edge rotation*, bei der ein Endpunkt der Kante unverändert bleibt. Die Rotation erfolgt also nur um einen Knoten. Die dritte Variante, der *edge slide*, schränkt die *edge rotation* weiter ein. Hier darf der veränderte Knoten nur ein Nachbar des ursprünglichen Knotens sein.



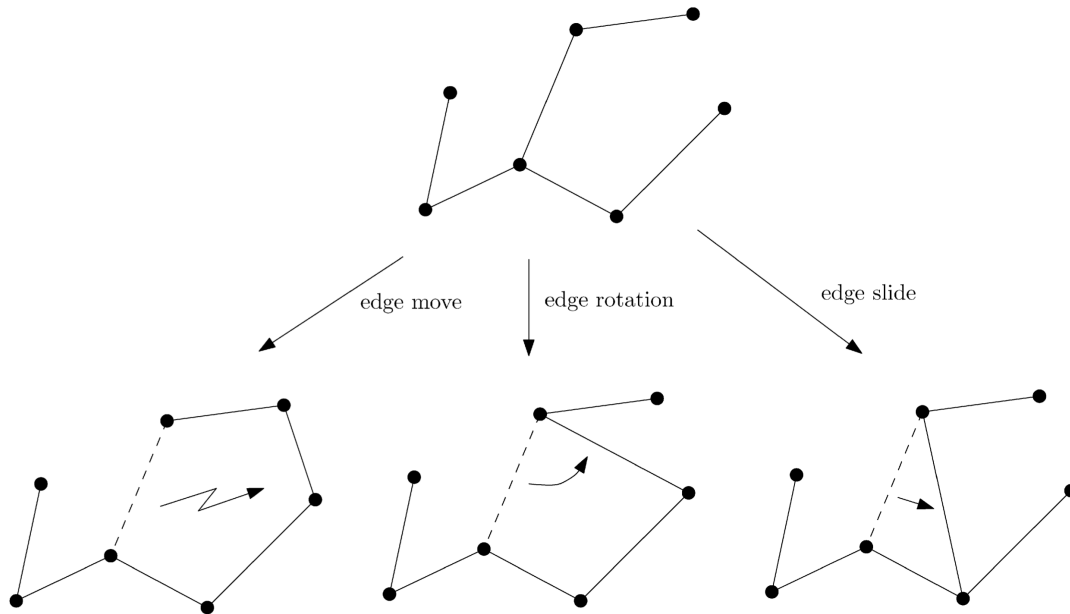


Figure 1.2: *Examples of edge move, edge rotation, and edge slide* aus [1, p. 70]

### 1.2.3 degree constrained minimum spanning tree

Eine Grad regulierung gibt es auch beim minimum spanning tree. Hier hat jede Kante ein Vorgegebenes Gewicht welches bei dem spanning tree minimiert werden soll. Das besondere am *degree constrained* ist das ein minimum spanning tree erzeugt werden soll, welcher an jedem Knoten einen maximalen Grad von  $d$  hat. Wobei  $d$  Global für das Problem vorgegeben wird.

Es wurden verschiedene Ansätze von Joshua Knowles und David Corne [4] vorgestellt, welche das Problem lösen sollen. Der erste Ansatz ignoriert den minimum Aspekt des Problems zu Anfang. Es werden Kanten hinzugefügt welche die Grad Begrenzung nicht verletzen. Und im laufe wird dann Probiert die Kanten zu minimieren.

Der minimale spanning Tree ohne Grad Begrenzung kann in polynomieller Zeit gelöst werden. Der nächste Ansatz erzeugt zuerst einen minimalen spanning Tree welcher die Grad Begrenzung ignoriert. Wenn eine Lösung gefunden wurde, wird den Kanten ein neues Gewicht zugeteilt. Dieses neue Gewicht wird mit folgender Formel berechnet.

$$newweight = weight + fault * max * \frac{(weight - min)}{max - min}$$

Wobei  $newweight$  das neue Gewicht ist,  $weight$  das aktuelle Gewicht. Die Variable  $fault$  hat den Wert  $\{0, 1, 2\}$ , abhängig davon wie viele Knoten an der Kante grade die Grad begrenzung verletzen. Und die Variablen  $min/max$  das kleinste/größe Gewicht aller Kanten. Danach wird wieder ein minimaler spanning Tree gebildet wobei durch

## 1 Introduction

das neue Gewicht jetzt Kanten bevorzugt werden welche keinen Grad verletzen. Diese beiden Schritt können so lange wiederholt werden bis eine Valide Lösung gefunden wurde oder eine Maximale Iteration erreicht wurde. Das Problem bei diesem Ansatz ist das Lösungen falsch als *nicht Möglich* Bewertet wurden, auf grund der Maximalen Iteration.

Ein weiter Ansatz wurde *hillclimbing* genannt. Dieser Löst das degree constrained minimum spanning tree Problem nicht, wird aber zur neuen Knoten suche in einem anderen Algorithmus genutzt. In diesem Ansatz werden lokal neue Knoten gesucht die bessere Ergebnisse liefern. Das könnte man für Degree constrained Triangulation anwenden um neue Kanten zu finden. Dieser Ansatz erzeugt keine eigene Lösung sondern verbessert eine Vorhandene. Es wird ein Zufälliger Knoten in einer Lösung ausgewählt. Danach werden weitere mögliche Knoten gesucht welche in der Nähe des Original Knotens sind. Bei diesen wird dann geschaut ob sie gleich oder bessere Ergebnisse liefern. Sollte das der Fall sein wird der Knoten ausgetauscht, sonst wird der gefundene Knoten ignoriert. Dieser Ansatz soll lokal gute Ergebnisse liefern, aber durch seine *greedy* Verhalten global keine Verbesserungen bringen. Eine mögliche Verbesserung für das Lokale Problem ist *multistart hillclimbing*. Hier wird nur ein Punkt in der Nähe ausgetestet. Sollte dieser nicht gleich oder besser sein, wird ein neuer Zufälliger Punkt ausgewählt.

Dieser Ansatz wird *simulated annealing* genannt. Auch hier werden wieder nur neue Knoten gesucht und vorhandene Lösungen verbessert. Der Ansatz will das Problem mit der Globalen Verbesserung lösen in dem er anfangs sehr viele Veränderung erlaubt und zum ende immer genauere Verbesserungen liefert. Auch hier wird wieder ein Random Knoten ausgewählt und weitere mögliche Knoten in der Nähe gesucht. Bei diesem Ansatz werden aber neue Knoten nach folgender Formel übernommen.

$$p\{\text{accept } j\} = \begin{cases} 1 & \text{if } f(j) \leq f(i) \\ \exp\left(\frac{f(i)-f(j)}{c}\right) & \text{sonst} \end{cases}$$

wo  $p\{\text{accept } j\}$  die Wahrscheinlichkeit ist das ein Knoten übernommen wird. Die aktuelle Lösung ist  $i$  und die mögliche neue Lösung ist  $j$ . Die Funktion  $f$  gibt hierbei die Kosten für die Lösung an. Der Parameter  $c$  ist Anfangs sehr hoch gesetzt und wird mit der Zeit dekrementiert. Das Verändern von  $c$  bewirkt die gewünschte Veränderungsrate. Der Ansatz endet wenn ein Finales  $c_f$  erreicht wurde, also  $c = c_f$ .

Der letzte Ansatz wird nur am Ende erwähnt und beschreibt das normale Verfahren für einen evolutionalgorithmus. Hierbei werden verschiedene Lösungen erstellt und bewertet. Die besten Lösungen werden ausgewählt und aus diesen Lösungen werden durch mutation neue Lösungen erstellt.

## 1.3 Overview

If needed, write down where to find everything.

## 2 Preliminaries

If needed, you can write down definitions here, that you need in your thesis. For this, the definition-environment can be helpful

**Definition 2.1** (Graph coloring). A graph  $G = (V, E)$  is  $k$ -colorable if there is a function  $C : V \rightarrow \{1, \dots, k\}$  such that  $C(u) \neq C(v)$  for any edge  $\{u, v\} \in E$

The *chromatic number* of a graph  $G$  is the smallest number  $k$  such that  $G$  is  $k$ -colorable.



## 3 Content

From here, start your main content. You can always use new chapters if needed. Use proper environments for your content, e.g. theorem, definition, lemma, etc. Here are some examples.

**Theorem 3.1.** *Every planar graph is 4-colorable.*

**Lemma 3.2.** *Every bipartite graph is 2-colorable.*

**Corollary 3.3.** *Every tree graph is 2-colorable.*

Because graphs with at least one edge need at least two colors, we obtain the following theorem.

**Theorem 3.4.** *The chromatic number of every (non-trivial) bipartite graph is 2.*



## 4 Experiments

If you have experiments, write down:

- Implementation details (e.g., libraries, third-party programs, ...)
- In which environment are you testing (e.g., system specifications)
- What are you testing? (e.g., runtime, space needed, fps, ...)
- What are the results? Show fancy figures!
- What does the results mean? (e.g., give a little discussion on your results)
- ...





## 5 Conclusion (and Future Work)

This is the end of your thesis! Give a summary of what you did.

You can also write down possible future work you or other people could look at.



# Bibliography

- [1] Prosenjit Bose and Ferran Hurtado. Flips in planar graphs. *Computational Geometry*, 42(1):60–80, 2009. URL: <https://www.sciencedirect.com/science/article/pii/S0925772108000370>, doi:10.1016/j.comgeo.2008.04.001.
- [2] Mark de Berg, Marc van Kreveld, Mark Overmars, and Otfried Cheong Schwarzkopf. *Delaunay Triangulations*, pages 183–210. Springer Berlin Heidelberg, Berlin, Heidelberg, 2000. doi:10.1007/978-3-662-04245-8\_9.
- [3] F. Hurtado, M. Noy, and J. Urrutia. Flipping edges in triangulations. In *Proceedings of the Twelfth Annual Symposium on Computational Geometry*, SCG '96, page 214–223, New York, NY, USA, 1996. Association for Computing Machinery. doi:10.1145/237218.237367.
- [4] J. Knowles and D. Corne. A new evolutionary approach to the degree-constrained minimum spanning tree problem. *IEEE Transactions on Evolutionary Computation*, 4(2):125–134, 2000. doi:10.1109/4235.850653.